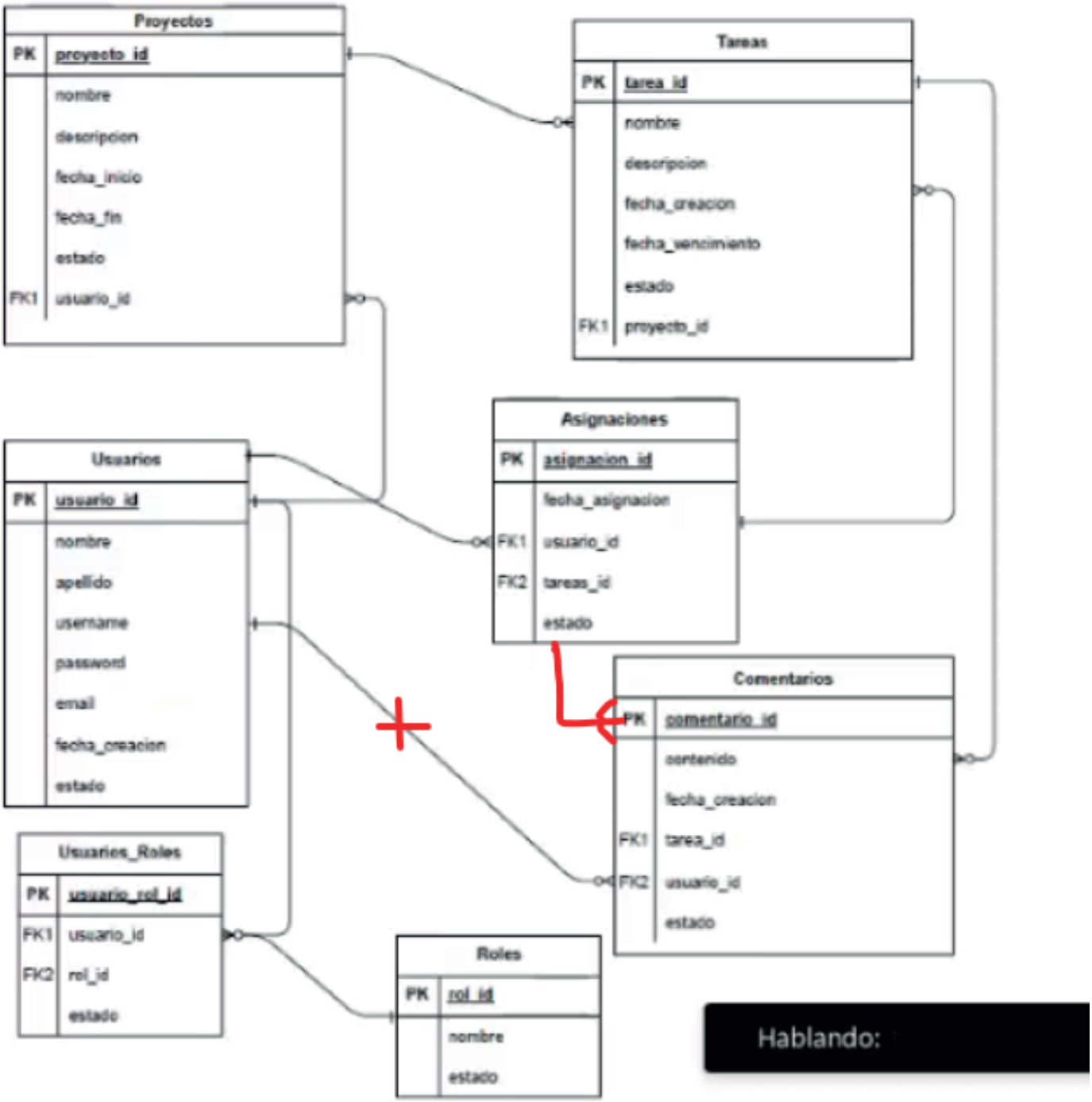
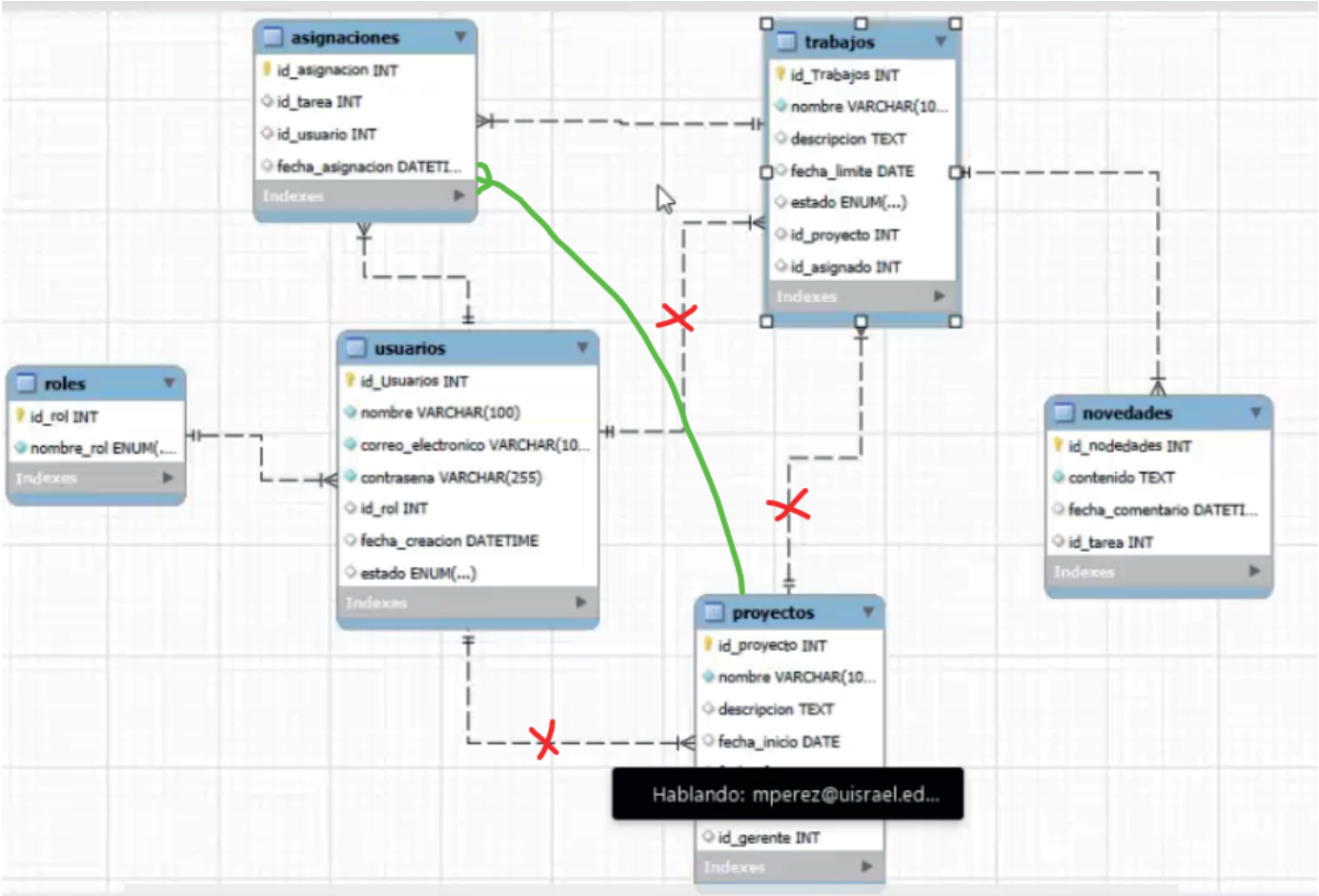


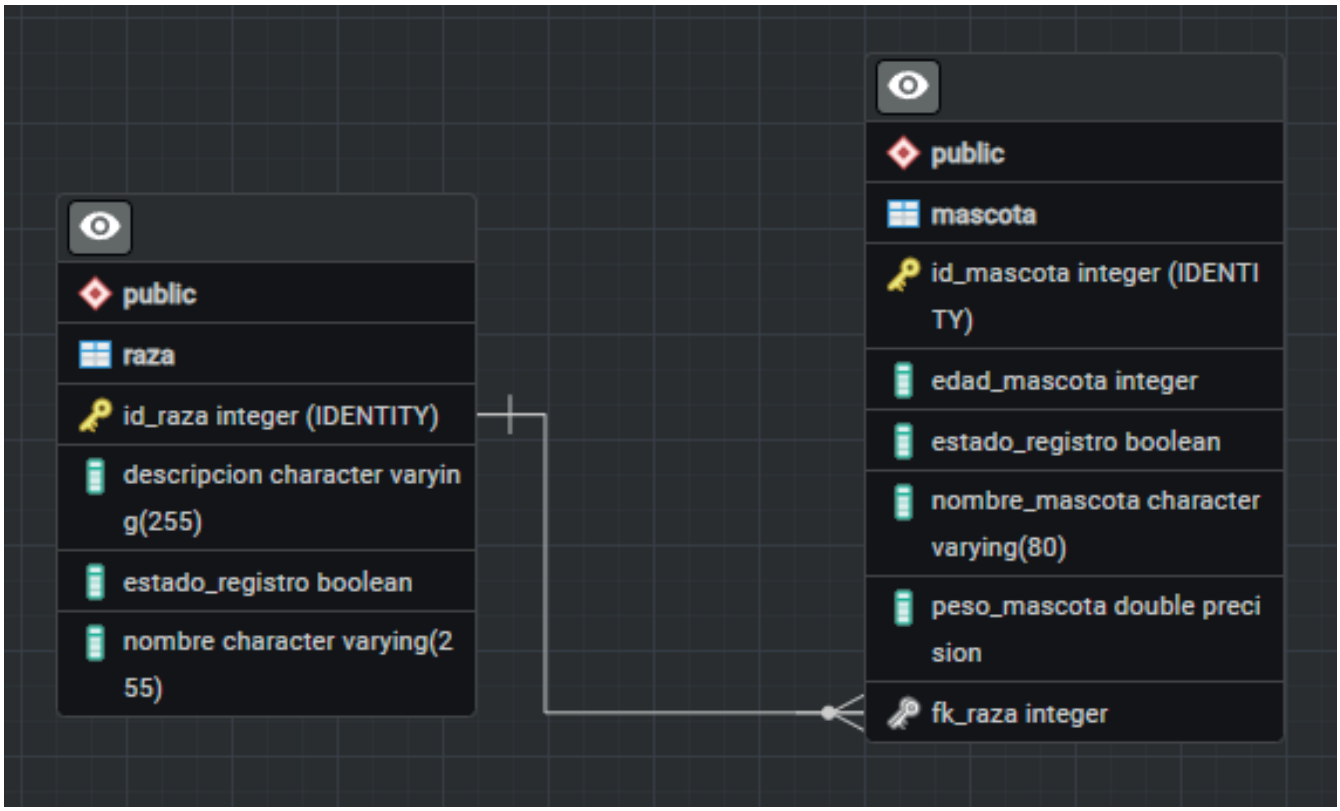
El módelo depende de la lógica de trabajo de la organización:

1. Lógica Básica: Cada actor tiene un rol específico, en un proyecto un usuario un rol especifico.
2. Dejar el Rol como tabla ya que no pertenece a la entidad Usuario
3. Relacionar Comentarios con asignaciones, “esa asignación tiene varios comentarios”



Correcto:
- Un trabajo(tarea) tiene varias novedades
- Varios usuarios pueden tener el mismo rol
- Un usuario puede tener asignado varias tareas (estado tarea: pendiente/terminada)
- El proyecto es el macro y las asignaciones deberían estan relacionadas al proyecto
- Relacionar el proyecto con asignaciones y Trabajos con asignaciones, por el usuario se accede a las asignaciones, tareas y proyectos

Mal:
- Eliminar relaciones duplicadas: Basta con un camino de comunicación para la relación.



Cardinalidad:
1 a muchos (1:N)

Una raza puede tener muchas mascotas.
Cada mascota pertenece a una sola raza.

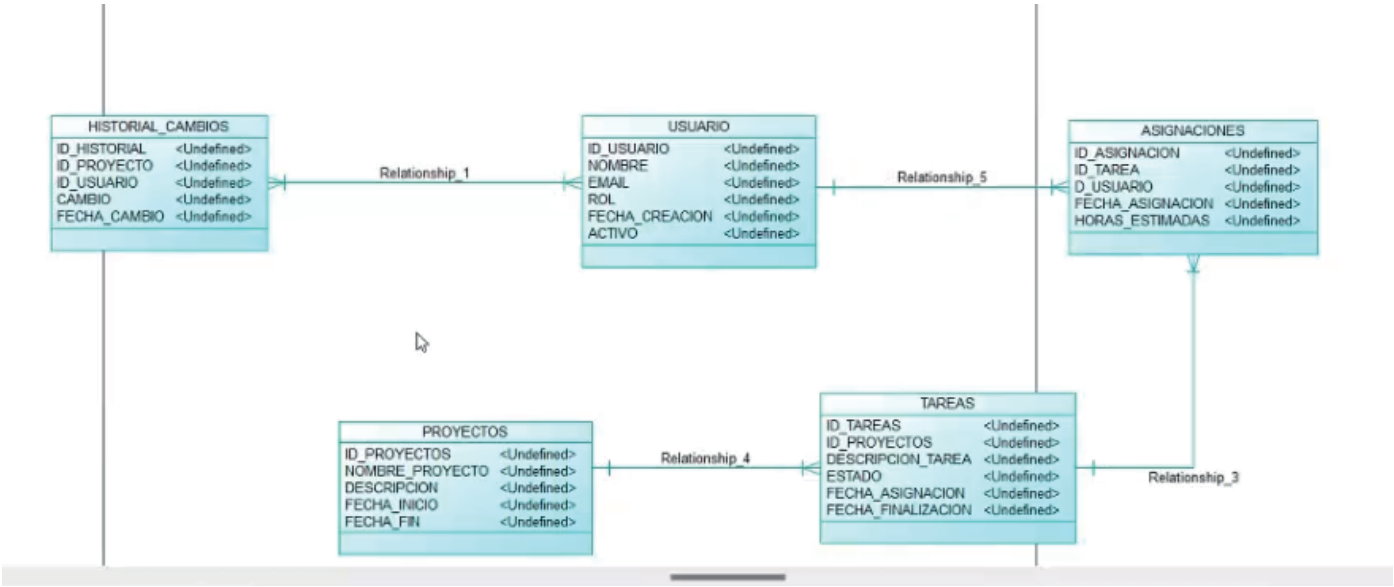
La tabla mascota tiene la clave foránea fk_raza.
Esa FK apunta a id_raza en la tabla raza.
En forma simple:
raza (1) -----> (N) mascota

Ejemplo:
Raza: Labrador puede haber muchas mascotas (Firulais, Max, Rocky…)
Pero cada mascota solo tiene una raza.

Relación dependiente (no recomendado)
Línea continua y cardinalidad
El fk es not null u obligatorio
fk pasa a formar parte de la pk (primary key compuesto)
- manejar bodegas por sucursales, diferentes sucursales y en cada una tiene diferentes bodegas y son obligatorias en cada sucursal

***Relación No dependiente** (con la que se trabaja más)
Línea entrecortada y cardinalidad
pk pasa como fk como un atributo más

La cardinalidad de muchos a muchos:
Se puede programar en java, pero es recomendable trabajar con tablas de ruptura.



Correcto:
- Un usuario tiene varias asignaciones
- Una tarea tiene varias asignaciones

Mal:
- La tabla principal debe diseñarse primero
- Relación de varios a varios no existe en un modelo lógico